

Formal verification for I²C communication Protocol in aerospace and aviation industries

Merve Berik^a, Yahya Baykal^{b,*}

^a TUSAŞ Türk Havacılık ve Uzay Sanayi, Fethiye mah. Havacılık Bulvarı, no: 17, Kahramankazan, Ankara, Turkey

^b Çankaya University, Department of Electrical-Electronics Engineering, Yukarıyurtcu mah. Mimar Sinan cad, No: 4, Etimesgut, 06815, Ankara, Turkey

ARTICLE INFO

Keywords:

Formal Verification
System Verilog
Aerospace
Hardware
Design

ABSTRACT

The aerospace industry comprises many safety-critical applications that involve a vast number of interacting subsystems. Reliable data communication between devices and components is therefore essential. In this context, Inter-Integrated Circuit (I²C) communication protocol is widely preferred due to its simplicity, flexibility, low power consumption, and reliability. However, issues such as data corruption, data loss, and increased latency may still occur and can lead to serious consequences in aviation, including safety risks, electronic malfunctions, air traffic management problems, and incorrect navigation information. To avoid such failures, the I²C Register-Transfer Level (RTL) design must be both correctly implemented and rigorously verified. There are several verification methods for digital design verification. Among several digital design verification approaches, Formal Verification (FV) is one of the most precise and reliable methods for safety-critical systems, as it provides mathematical proofs of conformance to specified properties. In this work, an open-source, Yosys-based formal verification flow is applied to an open-source I²C master design using the SymbiYosys framework. The verification environment is developed in SystemVerilog with SystemVerilog Assertions, enabling the detection of design errors directly against the protocol requirements. By combining bounded model checking, cover analysis, and theorem-proving, the proposed flow systematically verifies all five finite-state-machine (FSM) states and nine transitions of the I²C master. The results demonstrate that formal verification can systematically ensure robust and fault-tolerant I²C operation for avionics applications.

1. Introduction

Modern computer systems and hardware platforms are becoming increasingly complex, which makes it challenging to ensure their correctness and to detect subtle design errors. One effective way to address these challenges is to apply Formal Verification (FV) methods. FV relies on mathematically rigorous techniques that reason about the behavior of a design with respect to a set of formally specified properties. Previous simulation-based verification studies [1–4] have used SystemVerilog Assertions (SVA) and the Universal Verification Methodology (UVM) to reduce verification time and improve modular reuse. These approaches typically rely on constrained random test generation and sophisticated test benches, but they cannot exhaustively explore all reachable states of the design. In contrast, model checking-based formal verification systematically analyses the state space within the configured bounds and can provide stronger guarantees for safety-critical applications when combined with well-defined properties.

These methods are important for complex systems; however, they do not always provide significant time savings for large designs and require a detailed verification plan. Moreover, they cannot mathematically prove that all possible states and transitions satisfy the intended properties, nor can they guarantee that the register-transfer level (RTL) implementation matches all subsequent representations such as synthesis or scan.

Furthermore, all of these methods require manual creation of the test environment, including components such as test cases, drivers, monitors, models, and controllers, which increases both complexity and the risk of errors. As noted in [5], achieving time savings is therefore more difficult. Formal Verification (FV) is one of the approaches used to address these challenges and includes techniques such as theorem proving, equivalence checking, type checking, and model checking. In [6], FV is combined with validation methods to improve system quality and reliability.

In this work, an assertion-based verification (ABV) approach is

* Corresponding author.

E-mail address: y.baykal@cankaya.edu.tr (Y. Baykal).

<https://doi.org/10.1016/j.micpro.2026.105252>

Received 17 August 2025; Received in revised form 18 November 2025; Accepted 8 February 2026

Available online 20 February 2026

0141-9331/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

adopted, in which the intended behavior of the I²C master is expressed as properties written in the SystemVerilog Assertions (SVA) language. These SVA properties serve as the specification for our Yosys/SymbiYosys-based model-checking flow, enabling us to combine the expressiveness of SVA with the exhaustive analysis capabilities of formal verification for the safety-critical I²C protocol.

Recent works on formal verification in the literature has introduced assertion-based verification frameworks that use SystemVerilog Assertions to rapidly detect RTL bugs in I²C modules and enable reuse across simulation and formal flows [7].

Another study automatically synthesizes verified hardware/software drivers for I²C devices using model checking to ensure the driver and hardware jointly satisfy the protocol specification [8]. Emerging AI-assisted techniques have also begun to generate verification assertions directly from natural language specifications, although these methods are still maturing [9–11]. These developments highlight the growing interest in combining formal methods with other verification paradigms to improve coverage and efficiency. Furthermore, Formal verification tools have been increasingly adapted to domain-specific protocols, as shown for automotive CAN and [7] for avionics I²C, which demonstrates the scalability of our chosen methodology.

The main contribution of this paper is an applied case study of an open-source, Yosys/SymbiYosys-based formal verification flow for an I²C master controller in an aerospace context. The work documents (i) how a design and domain specific formal environment and assertion specification can be constructed for an open-source I²C master RTL that is intended to emulate a DO-254 / AMC 20-152A style avionics design, (ii) how a reusable set of safety and liveness oriented checks related to START/STOP timing, ACK handling and FSM progression can be derived from textual I²C requirements and analysed using BMC, Prove and Cover modes, and (iii) how the resulting PASS, FAIL and UNKNOWN outcomes, together with induction based proofs and coverage, can be interpreted in terms of practical avionics scenarios and development assurance considerations. To the authors' knowledge, there is limited published experience on using open-source formal verification flows for safety-critical I²C based communication in aerospace, and this paper aims to fill that gap as a detailed and reproducible experience report.

The rest of this paper is organized as follows. Section 2 presents the background of the I²C protocol in aerospace applications and states the objectives of the work. Section 3 describes the proposed formal verification methodology and the overall verification flow. Section 4 summarizes the specification of the I²C master digital design under verification. Section 5 details the formal verification framework, including the Formal Core, the verification methods, and the SymbiYosys-based tool configuration. Section 6 reports the formal verification results for the I²C master, including the BMC, Cover, and Prove experiments. Finally, Section 7 concludes the paper and outlines directions for future work.

2. Background and Objectives

This section first provides background information on the I²C communication protocol and its role in aerospace applications, and then states the specific objectives of this work.

The I²C protocol is a serial communication standard that facilitates data exchange between microcontrollers, electronic devices, and sensors over a two-wire bus. In avionics and other embedded applications, I²C is widely preferred due to its simplicity, flexibility, ease of use, and low cost. The correct operation of the I²C protocol and the reliability of data communication are of critical importance in aerospace systems. Failures such as data corruption, data loss, or excessive latency can lead to serious consequences, including malfunctioning sensor readings and incorrect control decisions. Therefore, the verification of I²C designs is a crucial part of the overall design process.

Several verification methods can be used for digital designs. In recent years, formal methods have become increasingly important,

demonstrating significant benefits in terms of verification depth and coverage. By providing mathematically based checks against formally specified properties, Formal Verification (FV) offers a high verification potential, particularly for safety-critical systems. For this reason, FV is the focus of this study, as also motivated by the work in [12].

In many previous studies, formal verification has typically been performed using commercial EDA tools. For instance, in [13], a Cadence formal tool is utilized. Although such tools are user friendly and well integrated into industrial flows, they are proprietary solutions and often involve significant licensing costs. Moreover, until recently there has been no widely adopted open-source formal verification toolchain reported for safety-critical communication protocols in aerospace. In this work, by contrast, the entire flow is based on open-source tools (Yosys and SymbiYosys), which eliminates license costs and reduces the budgetary barrier for adopting formal verification in avionics projects.

This paper focuses on applying formal verification methods to an I²C master digital design that plays a crucial role in device communication in the aviation industry. FV, which is based on mathematical techniques, is one of the most reliable verification approaches for critical systems, enabling early detection of design errors even in complex implementations, as discussed in [12].

Within the scope of this work, a SymbiYosys-based formal verification flow is applied to the RTL implementation of an open-source I²C master design described in [14], and the possible verification outcomes (PASS, FAIL, and UNKNOWN) for the finite-state machine (FSM) behavior are analysed. The primary objective of this study is to achieve a PASS outcome for each state defined in the FSM of the I²C master design by applying the formal verification methods under the configured bounds.

3. Methodology and Approach

This section describes the proposed formal verification methodology, including the overall flow, the verification environment, and the configuration of the SymbiYosys-based toolchain.

This study follows a formal verification (FV) approach to examine the relevant state space up to the configured bounds of the I²C master design systematically. I²C Master design, written in the Verilog Hardware Description Language (HDL) for this article, has been downloaded from the OpenCores website, where open-source hardware codes are available [15].

The verification process adopted in this work follows a method that differs from conventional approaches in the literature by creating a design specific formal environment. The key component of this environment is the Formal Core, in which the verification logic is developed using the SystemVerilog language and SVA directives. An example of a verification environment that employs SVA directives can be found in [16].

FV of the I²C RTL has been conducted using the Yosys-based SymbiYosys tool [17]. Additionally, this tool is free and open-source; however, it has some limitations. The most crucial and labor-intensive aspect of using this tool is that it does not support the entire Property Specification Language (PSL).

Creating a *.sby configuration file is necessary to employ the SymbiYosys tool and initiate the FV process. Based on this file, specifications such as which verification options (Bounded Model Checking (BMC), Cover, or Prove) will be employed, the number of steps to be performed, and the selection of the engine (yices 2, boolector, aiger etc.) are specified. These complementary methods balance between checking reachable states, finding counterexamples, and mathematically proving correctness.

To verify the finite state machine (FSM) of the I²C master design, the sby file used in this work defines macros such as WRITE, READ, WRITE_DIRECT and STOP. Each macro constrains the inputs and expected outputs so that the solver can focus on a single FSM state, and the macros are invoked separately for the BMC, Cover and Prove runs.

GTKWave is a waveform viewer software tool. In the formal verification of the I²C RTL, waveform files generated by FV tools are displayed, and the behavior and timing of the design are visually analyzed through this program.

4. Specification of the DUT- I²C Master Digital Design Submission

4.1. I²C Communication Procedure

I²C bus is a serial communication protocol that consists of two active lines: Serial Clock Line (SCL) and Serial Data Line (SDA). While actual data is carried on the SDA line, the SCL line is a synchronized clock signal shown in the article [18].

I²C communication protocol is widely preferred over other protocols due to its high speed, flexibility, low power consumption, sensitivity, and reliability.

Master device, depending on the slave device it will use, is the device that sends or requests data along with the slave address. Generally, communication in I²C starts with a START signal and is finished by a STOP signal [19]. READ, WRITE, START, and STOP, ACK (Acknowledge) conditions are used for data transfer.

START State:

When Master device wants to start data transfer, it sends a start condition. Then, it sends the slave device's address and the read or write bit for the desired operation. It sends a start condition by lowering the SDA line while the SCL line is high, shown in Fig. 1 in the Start State.

STOP State:

When Master device wants to finish data transfer, it sends a stop condition to inform the slave devices that the data transfer is complete. It sends a stop condition by raising the SDA line while the SCL line is high, shown in Fig. 1 in the Stop State.

ACK State:

This state is used to indicate that a device has successfully received data or to confirm the correctness of the transmitted data during data transmission. Fig. 1 shows the frame format of the I²C protocol according to the explanations above.

4.2. I²C Master Digital Design Specification

OpenCores, where the I²C Master Verilog design is located, also has an I²C Master specification document in [18]. Actually, this design is a Wishbone protocol feature design, but the Formal Core for this article was created according to the Finite State Machine document in [18]. Fig. 2 includes this Finite State Machine and shows the states that the Master should be in when data is read or written. These are the IDLE, START, WRITE, READ, and ACK states.

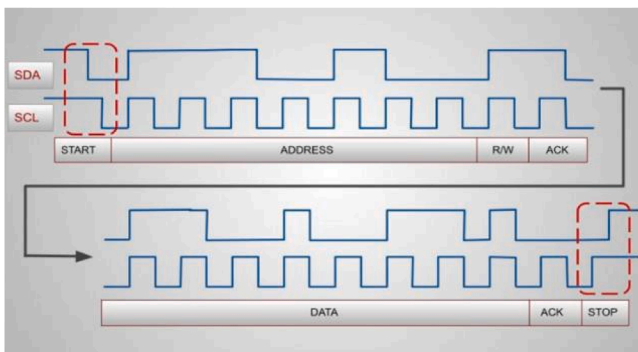


Fig. 1. I²C Frame Format.

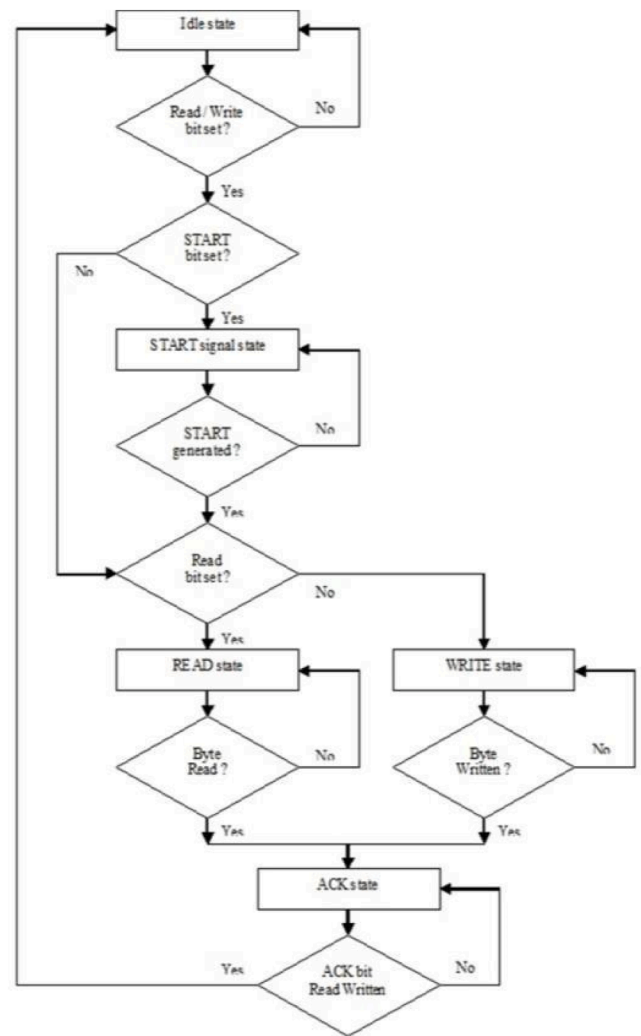


Fig. 2. Finite State Machine.

5. Formal verification

This section presents the formal verification framework, including the environment, the verification modes (BMC, Cover, Prove), the Formal Core, and the Yosys-based SymbiYosys tool.

5.1. Formal Verification Environment

Fig. 3 presents a flowchart that outlines the structure of the formal verification environment. The diagram highlights the main artifacts and tools used in the approach, including the I²C master RTL design, the Formal Core, the SymbiYosys/Yosys toolchain, the configuration file, and the resulting logs and waveform outputs. The verification logic in the Formal Core has been implemented in SystemVerilog based on the I²C specification. For the inputs of the DUT, constraint structures are created using the assume directive, whereas for the desired outputs of the DUT, checkers are written using the assert directive.

SymbiYosys must be configured to run the required verification modes (such as BMC, Cover, and Prove) together with method-specific parameters, including the bound depth, solver selection, and engine options.

In order to feed the tool for necessary configurations <config_name>.sby file need to create and pass to tool as an argument. Assertions are written according to the design specifications to obtain the output as "PASS", "FAIL", "UNKNOWN". The waveform results are observed using the GTKWave waveform viewer program.

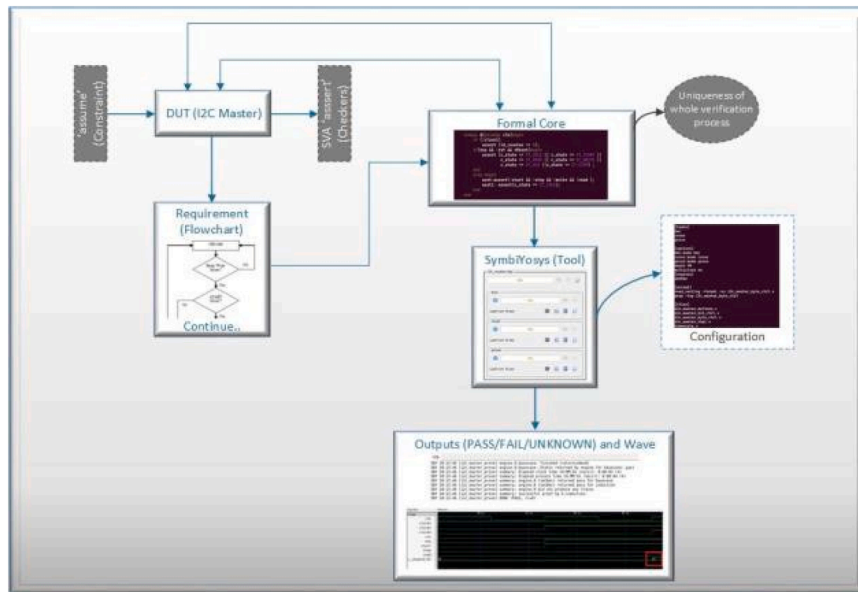


Fig. 3. Formal Verification Environment.

Formal Verification Environment shown in Fig. 3 is a unique framework that is applicable across all stages of design verification.

5.2. Formal Verification Methods

Three verification methods, namely BMC, Prove, and Cover have been employed in this work, as described in [20]. In this study, these methods are combined within a unique, design-specific verification framework, distinct from the more conventional flows reported in the literature, and can be selected and executed through the SymbiYosys tool.

5.2.1. Bounded Model Checking

Bounded Model Checking (BMC) explores all reachable states of the design up to a user-defined depth. At each depth it attempts to find counterexamples to the specified property; if a counterexample is found, the run reports FAIL, and if no counterexample is found within the given depth the result is PASS. Since the search is bounded, deeper corner-case bugs beyond the chosen depth cannot be detected, which is a known limitation.

5.2.2. Cover

The Cover method checks the reachability of a particular design state or condition within a specified depth. It verifies whether there exists an execution path that satisfies the given condition; if such a path is found, the tool reports a PASS result, otherwise it reports FAIL. Cover is a fast technique that provides evidence of achievability, but it does not guarantee correctness across all states and is therefore insufficient on its own. However, its disadvantage is that it may not cover all scenarios and may not provide a conclusive result.

5.2.3. Theorem Proving

It performs precise model checking by systematically exploring the behavior of the design within the configured bounds and provides mathematical evidence for the specified properties through induction-based reasoning. It first searches for a counterexample in a base depth and then attempts to establish an induction step. If it finds an error at any depth, it reports 'FAIL,' and if it doesn't find any error at any level, it reports 'PASS'; otherwise, if the induction cannot be completed the result is UNKNOWN. In the 'UNKNOWN' state, it indicates uncertainty about whether the design satisfies the property.

The Theorem Proving method checks the validity of an assertion at any depth through the Basecase and Induction phases: Basecase: It verifies whether the assertion is valid at a specific depth. The Basecase corresponds to the BMC method, and the result can be either 'PASS' or 'FAIL' based on the accuracy of the assertion.

5.3. Formal Core

The Formal Core consists of verification logic, which is described with SVA directives and SystemVerilog language. Several studies in [17, 21–23] demonstrate the effective use of SVA in similar contexts. After the necessary infrastructure was implemented, each state in the finite-state machine (FSM) was individually verified using these assertions. The Formal Core therefore serves as the central unit where the verification logic is defined and maintained.

In this environment, the intended behavior of the I²C master is captured through an assertion specification written as SystemVerilog immediate assertions, using the assert/assume/cover style supported by the Yosys/SymbiYosys Verilog back-end. Assertions are placed inside clocked processes that drive the FSM and I²C interface signals. Representative checks include:

- Safety: START/STOP timing: when a START condition is issued, SCL is high and SDA performs the required high-to-low transition; when a STOP condition is issued, SCL is high and SDA transitions low-to-high, returning the bus to the idle state. A violation may cause corrupted or misaligned transactions on the I²C bus in an avionics subsystem.
- Safety: ACK handling: after each data byte transfer, the receiver drives SDA low during the ACK bit unless a NACK is explicitly signalled. A violation may lead to undetected data loss between sensors, memory, or controllers.
- Liveness: Transaction completion: under valid input conditions and without error flags, the FSM is not allowed to remain permanently in intermediate states and must eventually reach a completion or idle state. A violation may indicate a bus lock-up or a hung controller, which can prevent timely delivery of critical sensor data.

These checks are implemented as SystemVerilog assert/assume/cover statements consistent with the patterns described in the SymbiYosys documentation for Verilog-based formal verification.

For readers with a more theoretical background, these checks can also be viewed in a temporal-logic style. Informally, the ACK handling requirement corresponds to “whenever a data byte is transferred, an ACK follows unless a NACK is intended”, and the transaction-completion requirement corresponds to “whenever a valid transaction starts, a completion or idle state is eventually reached under normal conditions”.

According to specifications ‘assume’ (Constraint) and ‘assert’ (Checkers) statements have been written. The ‘assume’ directive is used to create constraint structures with the inputs of the DUT. It makes the State Space smaller. The ‘assert’ directive is used to write checkers for the desired outputs of the DUT.

When ‘assume’ (Constraint) and ‘assert’ (Checkers) structures are used, at the background, the tool generates algorithmic models to realize these structures. The tool analyzes each checker of the DUT and searches for an input that can prove the checker wrong. If it finds such a path, it reports the error scenario. Moreover, ‘cover’ statement has been written to find out whether an assertion is reached or not.

5.4. Yosys-Based SymbiYosys Tool

SymbiYosys has a structure that increases system reliability and quality, saves time, facilitates system understandability and maintenance thanks to its modular structure support, and enables different components of the system to be used together and interact with each other.

To initiate the formal verification process and use the SymbiYosys tool, a configuration file with a *.sby extension has been created. Based on this file, it specifies which mode to use, either “BMC”, “Cover” or “Prove”, how many steps to perform, and which engine to select. Additionally, it includes commands for reading and preparing the design file to be verified according to its features.

SymbiYosys (SBY) is a front-end driver for Yosys-based formal hardware verification. It reads the .sby configuration file, invokes Yosys to generate the verification model, and coordinates the back-end engines and Satisfiability Modulo Theories (SMT) solvers to run BMC, proof, and cover checks on the specified assertions.

The tool adds the inputs, outputs the DUT that affect the checker to a variable set called Cone-of-Influence (COI). After adding all variables and determining the COI, the tool determines the state space of possible values. It then checks each path in the state space in parallel to verify whether the assertion is true. This process proves the compatibility of the design with the checker. The installation and usage of this tool followed the guidelines in the Yosys manual [20].

6. Formal verification results

The test scenarios were created with the purpose of verifying the I²C Master design’s “START”, “STOP”, “WRITE”, “READ”, “IDLE” and “ACK” states based on their respective inputs and outputs.

For the verification code written in SystemVerilog, the necessary infrastructure was first implemented and the clock pulses that enable transitions between states were generated. At the first clock pulse, the initially high reset signal is driven low to activate all other signals. Subsequently, the required assignments for the active signals were applied in accordance with the specification.

After this setup, the verification logic was implemented as SystemVerilog assertions to specify the conditions under which the system should reside in each FSM state, based on the initial conditions described in Section 4.2. The behavior of each state in the FSM was then verified using the SymbiYosys tool, and the results were reported as PASS. In addition, fault-injection scenarios were created by deliberately introducing incorrect conditions for properties that previously held, in order to observe how the system behaves when FAIL or UNKNOWN outcomes are produced by inverting the corresponding logic.

“Protocol Checking” has been performed for Cover and BMC methods, but it is not supported for the Prove method. Therefore, this

checking has not been done for Prove.

Transitions between states in the FSM Flow (Fig. 2) and the steps at which they are reached for this study are illustrated in Fig. 4. The reason ‘Stop State’ does not appear in State Space in Fig. 4 is that the FSM Flow of I²C Master Digital Design (Fig. 2) does not include the Stop State.

6.1. Bounded Model Checking

Using bounded model checking, the reachable state space of the design is explored up to a specified depth. Within this bound, all states and transitions encountered that are relevant to the verified properties are checked.

6.1.1. Start State

According to the specifications in the FSM, all ‘ST_START’ states (Start to Read, Start to Write etc.) have been checked, and consequently, this state has been verified.

Before the ‘ST_START’ state, the results for the ‘Read/write bit set?’ condition are provided Fig. 5 and the waveform in Fig. 6.

6.1.2. Write State

According to the specifications in the FSM, all ST_WRITE states have been fulfilled, and consequently, this state has been verified.

All ST_WRITE transitions (WRITE to WRITE and WRITE to ACK) were verified up to 300 steps and yielded “PASS” results “PASS” and possible “FAIL” statuses for the “WRITE to ACK” state are provided Fig. 7 and Fig. 8. In Fig. 7, if (v_clk_cnt_ack > 2) assert (c_state == ST_ACK).

In Fig. 8, asserting ‘c_state’ ≠ “ST_ACK” when (v_clk_cnt_ack > 2) produces a “FAIL” result, showing the assertion is incorrect. In reality, the system is in the expected “ST_ACK” state (h08) after “ST_WRITE” (h04), as confirmed by the waveform in Fig. 9.

6.1.3. Read State

According to the specifications in the FSM, all ST_READ states have been fulfilled, and consequently, this state has been verified.

All ST_READ transitions (READ to READ and READ to ACK) were verified up to 300 steps, yielding “PASS” results, with the “READ to ACK” status provided similarly to Fig. 7.

6.1.4. ACK to IDLE State

According to the FSM specifications, the ‘ACK to IDLE’ condition was verified within 330 steps, yielding a “PASS” result as shown in Fig. 10, with the waveform in Fig. 11 confirming this state.

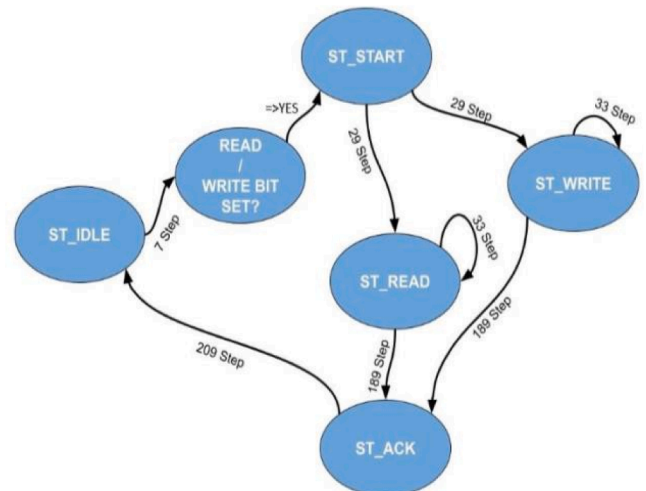


Fig. 4. State Space.

```

171 2:12:55 [i2c_master_bmc] engine_0: ## 0:00:16 Checking assertions in step 86..
172 2:12:56 [i2c_master_bmc] engine_0: ## 0:00:17 Checking assertions in step 87..
173 2:12:56 [i2c_master_bmc] engine_0: ## 0:00:17 Checking assertions in step 88..
174 2:12:56 [i2c_master_bmc] engine_0: ## 0:00:18 Checking assertions in step 89..
175 2:12:57 [i2c_master_bmc] engine_0: ## 0:00:18 Checking assertions in step 90..
176 2:12:57 [i2c_master_bmc] engine_0: ## 0:00:18 Checking assertions in step 91..
177 2:12:58 [i2c_master_bmc] engine_0: ## 0:00:19 Checking assertions in step 92..
178 2:12:58 [i2c_master_bmc] engine_0: ## 0:00:19 Checking assertions in step 93..
179 2:12:59 [i2c_master_bmc] engine_0: ## 0:00:20 Checking assertions in step 94..
180 2:12:59 [i2c_master_bmc] engine_0: ## 0:00:20 Checking assertions in step 95..
181 2:13:00 [i2c_master_bmc] engine_0: ## 0:00:21 Checking assertions in step 96..
182 2:13:00 [i2c_master_bmc] engine_0: ## 0:00:21 Checking assertions in step 97..
183 2:13:01 [i2c_master_bmc] engine_0: ## 0:00:22 Checking assertions in step 98..
184 2:13:01 [i2c_master_bmc] engine_0: ## 0:00:22 Checking assertions in step 99..
185 2:13:01 [i2c_master_bmc] engine_0: ## 0:00:22 Status: passed
186 2:13:01 [i2c_master_bmc] engine_0: finished (returncode=0)
187 2:13:01 [i2c_master_bmc] engine_0: Status returned by engine: pass
188 2:13:01 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:23 (23)
189 2:13:01 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:27 (27)
190 2:13:01 [i2c_master_bmc] summary: engine_0 (smtbmc --nopreset --unroll --keep-going) returned pass
191 2:13:01 [i2c_master_bmc] summary: engine_0 did not produce any traces
192 2:13:01 [i2c_master_bmc] DONE (PASS, rc=0)

```

Fig. 5. BMC Method – Start State, “PASS” Log.



Fig. 6. Waveform Result – BMC Method – Start State.

6.1.5. Stop State

According to the specifications in the FSM, after all condition checked, ‘STOP’ conditions have been checked within 150 steps. The “PASS” result has been observed in the Fig. 12, and the state verified.

6.1.6. Protocol Checking

6.1.6.1. Protocol Checking – START State. The start condition—lowering the SDA line while SCL is high, per the I²C frame format in Fig. 1—was verified in the Formal Core using the ‘WRITE’ and ‘PROTOCOL_CHECKING_START’ macros together in the configuration file, yielding a “PASS” result with bounded model checking within 250 steps (Fig. 13).

6.1.6.2. Protocol Checking – STOP State. The stop condition—raising the SDA line while SCL is high, per the I²C frame format in Fig. 1—was verified in the Formal Core using the ‘STOP’ and ‘PROTOCOL_CHECKING_STOP’ macros together in the configuration file, achieving a “PASS” result with bounded model checking within 250 steps, similar to the case shown in Fig. 13.

```

377 SBY 23:49:03 [i2c_master_bmc] engine_0: ## 0:02:39 Checking assertions in step 292..
378 SBY 23:49:04 [i2c_master_bmc] engine_0: ## 0:02:40 Checking assertions in step 293..
379 SBY 23:49:05 [i2c_master_bmc] engine_0: ## 0:02:41 Checking assertions in step 294..
380 SBY 23:49:06 [i2c_master_bmc] engine_0: ## 0:02:42 Checking assertions in step 295..
381 SBY 23:49:07 [i2c_master_bmc] engine_0: ## 0:02:43 Checking assertions in step 296..
382 SBY 23:49:08 [i2c_master_bmc] engine_0: ## 0:02:44 Checking assertions in step 297..
383 SBY 23:49:09 [i2c_master_bmc] engine_0: ## 0:02:45 Checking assertions in step 298..
384 SBY 23:49:10 [i2c_master_bmc] engine_0: ## 0:02:46 Checking assertions in step 299..
385 SBY 23:49:12 [i2c_master_bmc] engine_0: ## 0:02:47 Status: passed
386 SBY 23:49:12 [i2c_master_bmc] engine_0: finished (returncode=0)
387 SBY 23:49:12 [i2c_master_bmc] engine_0: Status returned by engine: pass
388 SBY 23:49:12 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:02:49 (169)
389 SBY 23:49:12 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:03:04 (184)
390 SBY 23:49:12 [i2c_master_bmc] summary: engine_0 (smtbmc --nopreset --unroll --keep-going) returned pass
391 SBY 23:49:12 [i2c_master_bmc] summary: engine_0 did not produce any traces
392 SBY 23:49:12 [i2c_master_bmc] DONE (PASS, rc=0)

```

Fig. 7. BMC-Write to Ack Condition, “PASS” Log.

```

396 SBY 0:11:00 [i2c_master_bmc] engine_0: ## 0:03:06 Checking assertions in step 292..
397 SBY 0:11:01 [i2c_master_bmc] engine_0: ## 0:03:07 Checking assertions in step 293..
398 SBY 0:11:02 [i2c_master_bmc] engine_0: ## 0:03:08 Checking assertions in step 294..
399 SBY 0:11:03 [i2c_master_bmc] engine_0: ## 0:03:09 Checking assertions in step 295..
400 SBY 0:11:04 [i2c_master_bmc] engine_0: ## 0:03:10 Checking assertions in step 296..
401 SBY 0:11:05 [i2c_master_bmc] engine_0: ## 0:03:11 Checking assertions in step 297..
402 SBY 0:11:06 [i2c_master_bmc] engine_0: ## 0:03:12 Checking assertions in step 298..
403 SBY 0:11:07 [i2c_master_bmc] engine_0: ## 0:03:13 Checking assertions in step 299..
404 SBY 0:11:08 [i2c_master_bmc] engine_0: ## 0:03:14 Status: failed
405 SBY 0:11:08 [i2c_master_bmc] engine_0: finished (returncode=1)
406 SBY 0:11:08 [i2c_master_bmc] engine_0: Status returned by engine: FAIL
407 SBY 0:11:08 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:03:15 (195)
408 SBY 0:11:08 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:03:31 (211)
409 SBY 0:11:08 [i2c_master_bmc] summary: engine_0 (smtbmc --nopreset --unroll --keep-going) returned FAIL
410 SBY 0:11:08 [i2c_master_bmc] summary: counterexample trace: i2c_master_bmc/engine_0/trace0.vcd
411 SBY 0:11:08 [i2c_master_bmc] summary: failed assertion i2c_master_byte_ctrl.st_ast_write2ack at i2c_n
412 SBY 0:11:08 [i2c_master_bmc] DONE (FAIL, rc=2)

```

Fig. 8. BMC – Write to Ack, Possible “FAIL” Log.



Fig. 9. Waveform Result – Write to Ack Condition.

```

407 SBY 21:27:11 [i2c_master_bmc] engine_0: ## 0:03:02 Checking assertions in step 322..
408 SBY 21:27:12 [i2c_master_bmc] engine_0: ## 0:03:03 Checking assertions in step 323..
409 SBY 21:27:14 [i2c_master_bmc] engine_0: ## 0:03:05 Checking assertions in step 324..
410 SBY 21:27:15 [i2c_master_bmc] engine_0: ## 0:03:06 Checking assertions in step 325..
411 SBY 21:27:16 [i2c_master_bmc] engine_0: ## 0:03:07 Checking assertions in step 326..
412 SBY 21:27:17 [i2c_master_bmc] engine_0: ## 0:03:08 Checking assertions in step 327..
413 SBY 21:27:18 [i2c_master_bmc] engine_0: ## 0:03:09 Checking assertions in step 328..
414 SBY 21:27:19 [i2c_master_bmc] engine_0: ## 0:03:11 Checking assertions in step 329..
415 SBY 21:27:21 [i2c_master_bmc] engine_0: ## 0:03:12 Status: passed
416 SBY 21:27:21 [i2c_master_bmc] engine_0: finished (returncode=0)
417 SBY 21:27:21 [i2c_master_bmc] engine_0: Status returned by engine: pass
418 SBY 21:27:21 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:03:13 (193)
419 SBY 21:27:21 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:03:25 (205)
420 SBY 21:27:21 [i2c_master_bmc] summary: engine_0 (smtbmc --nopreset --unroll --keep-going) returned pass
421 SBY 21:27:21 [i2c_master_bmc] summary: engine_0 did not produce any traces
422 SBY 21:27:21 [i2c_master_bmc] DONE (PASS, rc=0)

```

Fig. 10. BMC Method – Ack to Idle, “PASS” Log.

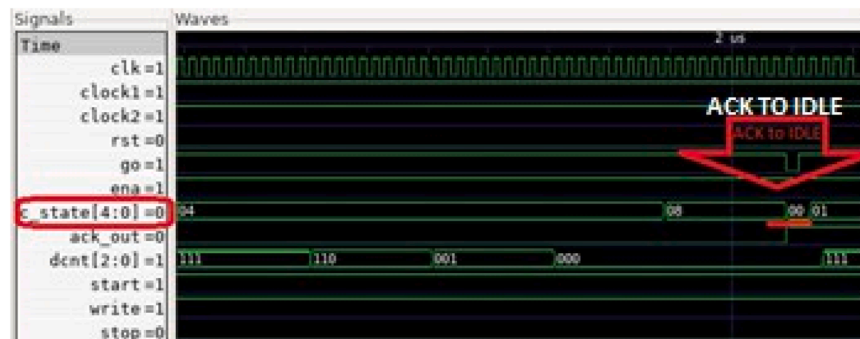


Fig. 11. Waveform Result – BMC Method – Ack to Idle.

```

261 SBY 16:27:55 [i2c_master_bmc] engine_0: ## 0:00:24 Checking assertions in step 148..
262 SBY 16:27:55 [i2c_master_bmc] engine_0: ## 0:00:25 Checking assertions in step 149..
263 SBY 16:27:55 [i2c_master_bmc] engine_0: ## 0:00:25 Status: passed
264 SBY 16:27:55 [i2c_master_bmc] engine_0: finished (returncode=0)
265 SBY 16:27:55 [i2c_master_bmc] engine_0: Status returned by engine: pass
266 SBY 16:27:55 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:26 (26)
267 SBY 16:27:55 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:33 (33)
268 SBY 16:27:55 [i2c_master_bmc] summary: engine_0 (smtbmc --nopreset --unroll --keep-going) returned pass
269 SBY 16:27:55 [i2c_master_bmc] summary: engine_0 did not produce any traces
270 SBY 16:27:55 [i2c_master_bmc] DONE (PASS, rc=0)

```

Fig. 12. BMC Method – Stop Condition, “PASS” Log.

6.2. Cover

According to the FSM, Cover method has been employed to determine whether the design reaches a state that satisfies a specific property.

6.2.1. Start State

According to the FSM specifications, all ‘ST_START’ states (‘START to WRITE’ and ‘START to READ’) were reached, with “PASS” results

observed within 400 steps.

6.2.2. Write State

According to the FSM specifications, Figs 14 and 15 demonstrate that all ‘ST_WRITE’ states (‘WRITE to WRITE’, ‘IDLE to WRITE’, ‘START to WRITE’, and ‘WRITE to ACK’) were successfully reached, with “PASS” results obtained. The ‘IDLE to WRITE’ state was achieved in 7 steps, as shown in Fig. 14, and its Cover result can be generated by defining the


```

345 SBY 17:34:19 [i2c_master_bmc] engine_0: ## 0:01:32 Checking assertions in step 248..
346 SBY 17:34:20 [i2c_master_bmc] engine_0: ## 0:01:32 Checking assertions in step 249..
347 SBY 17:34:20 [i2c_master_bmc] engine_0: ## 0:01:33 Status: passed
348 SBY 17:34:21 [i2c_master_bmc] engine_0: finished (returncode=0)
349 SBY 17:34:21 [i2c_master_bmc] engine_0: Status returned by engine: pass
350 SBY 17:34:21 [i2c_master_bmc] summary: Elapsed clock time [H:MM:SS (secs)]: 0:01:34 (94)
351 SBY 17:34:21 [i2c_master_bmc] summary: Elapsed process time [H:MM:SS (secs)]: 0:01:47 (107)
352 SBY 17:34:21 [i2c_master_bmc] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass
353 SBY 17:34:21 [i2c_master_bmc] summary: engine_0 did not produce any traces
354 SBY 17:34:21 [i2c_master_bmc] DONE (PASS, rc=0)

```

Fig. 13. BMC – Protocol Checking Start, “PASS” Log.

```

130 engine_0: ## 0:00:00 Appending additional step 16.
131 engine_0: ## 0:00:00 Appending additional step 17.
132 engine_0: ## 0:00:00 Re-solving with appended steps..
133 engine_0: ## 0:00:00 Reached cover statement at state_cov_idle2write in step 7.
134 engine_0: ## 0:00:00 Writing trace to VCD file: engine_0/trace0.vcd
135 engine_0: ## 0:00:01 Writing trace to Verilog testbench: engine_0/trace0.tb.v
136 engine_0: ## 0:00:01 Writing trace to constraints file: engine_0/trace0.smtc
137 engine_0: ## 0:00:01 Writing trace to Yosys witness file: engine_0/trace0.yw
138 engine_0: ## 0:00:01 Status: passed
139 engine_0: finished (returncode=0)
140 engine_0: Status returned by engine: pass
141 summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:02 (2)
142 summary: Elapsed process time [H:MM:SS (secs)]: 0:00:02 (2)
143 summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass
144 summary: cover trace: i2c_master_cover/engine_0/trace0.vcd
145 summary: reached cover statement i2c_master_byte_ctrl.state_cov_idle2write at i2c_master_byte_ctrl
146 DONE (PASS, rc=0)

```

Fig. 14. Cover – Idle to Write Condition, “PASS” Log.

```

316 engine_0: ## 0:00:43 Appending additional step 198.
317 engine_0: ## 0:00:43 Appending additional step 199.
318 engine_0: ## 0:00:43 Re-solving with appended steps..
319 engine_0: ## 0:00:43 Reached cover statement at st_cov_write2ack in step 189.
320 engine_0: ## 0:00:44 Writing trace to VCD file: engine_0/trace1.vcd
321 engine_0: ## 0:00:47 Writing trace to Verilog testbench: engine_0/trace1.tb.v
322 engine_0: ## 0:00:47 Writing trace to constraints file: engine_0/trace1.smtc
323 engine_0: ## 0:00:47 Writing trace to Yosys witness file: engine_0/trace1.yw
324 engine_0: ## 0:00:51 Status: passed
325 engine_0: finished (returncode=0)
326 engine_0: Status returned by engine: pass
327 summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:52 (52)
328 summary: Elapsed process time [H:MM:SS (secs)]: 0:01:04 (64)
329 summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass
330 summary: cover trace: i2c_master_cover/engine_0/trace0.vcd
331 summary: reached cover statement i2c_master_byte_ctrl.cov_write at i2c_master_byte_ctrl.v:505.4-505.100 in step 29
332 summary: reached cover statement i2c_master_byte_ctrl.st_cov_write2write at i2c_master_byte_ctrl.v:510.4-510.105 in step 29
333 summary: reached cover statement i2c_master_byte_ctrl.state_cov_idle2write at i2c_master_byte_ctrl.v:495.4-495.76 in step 29
334 summary: reached cover statement i2c_master_byte_ctrl.state_cov_start2write at i2c_master_byte_ctrl.v:501.4-502.59 in step 29
335 summary: cover trace: i2c_master_cover/engine_0/trace1.vcd
336 summary: reached cover statement i2c_master_byte_ctrl.st_cov_write2ack at i2c_master_byte_ctrl.v:514.4-514.99 in step 189
337 DONE (PASS, rc=0)

```

Fig. 15. Cover – Write Conditions, “PASS” Log.

“WRITE_DIRECT” macro in the *.sby configuration file. At the 189th step, the state was $ST_ACK = C_State$, as illustrated in Fig. 15, and in practice, the system was in the expected ‘ ST_ACK ’ state, as confirmed in Fig. 16.

6.2.3. Read State

According to the specifications in the FSM, All READ-related transitions were checked up to 400 steps; the design reached the ST_READ state and transitions such as “READ to READ”, “IDLE to READ”, “START to READ” and “READ to ACK” were observed with PASS results. “if

(v_clk_cnt_ack > 2) cover (c_state == ST_ACK)”.

When the condition “c.state” is written to be equal to a state other than “ ST_ACK ”, a “FAIL” result is obtained, indicating that this assertion is unreachable. In reality, the system is in the expected “ ST_ACK ” state, as observed in the waveform in Fig. 17. The “FAIL” result is obtained with the assertion “if(v_clk_cnt_ack > 2) cover (c_state != ST_ACK)”.

6.2.4. ACK to IDLE State

According to the FSM specifications, Fig. 18 shows the ‘ACK to IDLE’ state reaching step 209 with a “PASS” result, while Fig. 19 presents its

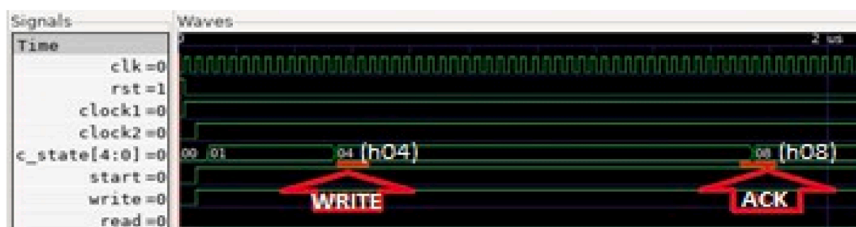


Fig. 16. Waveform Result – Write to Ack Condition.



Fig. 17. Waveform Result-Cover Method-Read to Ack.

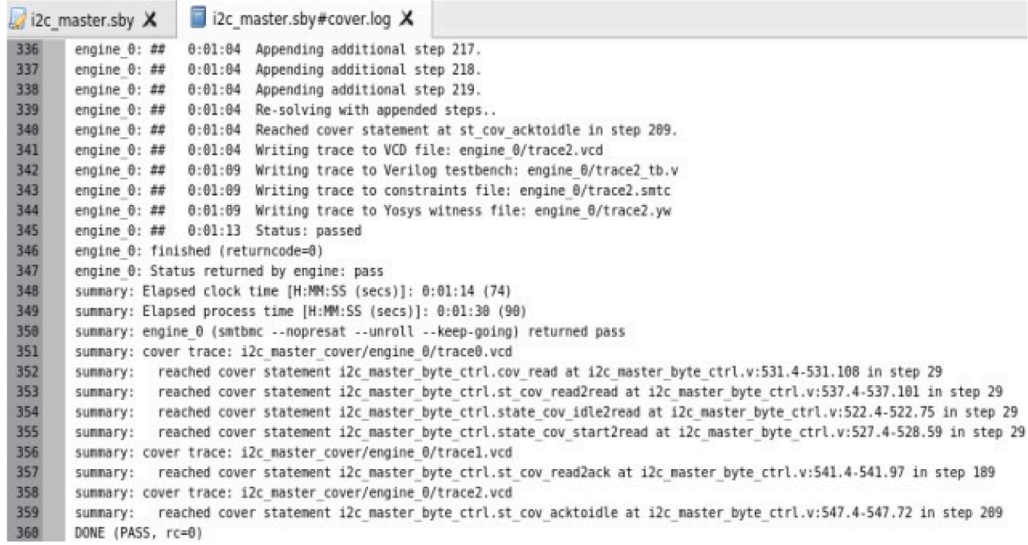


Fig. 18. Cover – Ack to Idle Condition, “PASS” Log.

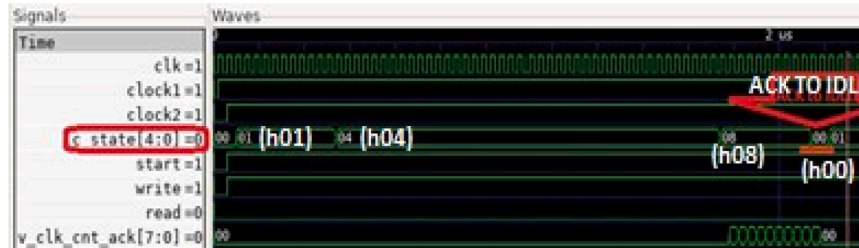


Fig. 19. Waveform Result – Ack to Idle Condition.

waveform.

6.2.5. Stop State

According to the FSM specifications, the ‘STOP’ states were reached with a “PASS” result, as shown in Fig. 20 at steps 9 and 27.

6.2.6. Protocol Checking

6.2.6.1. Protocol Checking – START State. The start condition—lowering SDA while SCL is high, as in the I²C frame format in Fig. 1—was verified in the Formal Core using the ‘WRITE’ and ‘PROTOCOL_CHECKING_START’ macros together, achieving a “PASS” around the 21st step (Fig. 21), with the waveform in Fig. 22 showing SDA and SCL initiation in the Start State.

6.2.6.2. Protocol Checking – STOP State. The stop condition—raising SDA while SCL is high, as in the I²C frame format in Fig. 1—was verified in the Formal Core using the ‘STOP’ and ‘PROTOCOL_CHECKING_STOP’

macros together, achieving a “PASS” around the 25th step (Fig. 23), with the waveform in Fig. 24 showing SDA and SCL termination in the Stop State.

6.3. Theorem Proving

The “Prove” method uses induction-based reasoning to attempt to establish that the specified properties hold for all time steps, distinguishing it from the “BMC” method, which only checks the design behavior up to a user-defined bounded depth.

When the Prove method was executed for the WRITE scenario, the initial result was “UNKNOWN”: the base case passed, but the induction step failed. Analysis of the counterexample waveform showed that the induction phase explored input combinations and counter values that are not realistic for the intended I²C operation. To resolve this, the constraint structures and Prove-task configuration were slightly strengthened by bounding auxiliary counters to their operational ranges and constraining the control signals to follow valid I²C protocol

```

i2c_master.sby X i2c_master.sby#bmc.log X i2c_master.sby#cover.log X i2c_master.sby#prove.log X
142 engine_0: ## 0:00:01 Checking cover reachability in step 22..
143 engine_0: ## 0:00:01 Checking cover reachability in step 23..
144 engine_0: ## 0:00:01 Checking cover reachability in step 24..
145 engine_0: ## 0:00:01 Checking cover reachability in step 25..
146 engine_0: ## 0:00:02 Checking cover reachability in step 26..
147 engine_0: ## 0:00:02 Checking cover reachability in step 27..
148 engine_0: ## 0:00:02 Reached cover statement at cov_stop2idle in step 27.
149 engine_0: ## 0:00:02 Writing trace to VCD file: engine_0/trace1.vcd
150 engine_0: ## 0:00:02 Writing trace to Verilog testbench: engine_0/trace1.tb.v
151 engine_0: ## 0:00:02 Writing trace to constraints file: engine_0/trace1.smtc
152 engine_0: ## 0:00:02 Writing trace to Yosys witness file: engine_0/trace1.yw
153 engine_0: ## 0:00:03 Status: passed
154 engine_0: finished (returncode=0)
155 engine_0: Status returned by engine: pass
156 summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:03 (3)
157 summary: Elapsed process time [H:MM:SS (secs)]: 0:00:04 (4)
158 summary: engine_0 (smtbmc --nopsat --unroll --keep-going) returned pass
159 summary: cover trace: i2c_master_cover/engine_0/trace0.vcd
160 summary: reached cover statement i2c_master_byte_ctrl.cov1_stop at i2c_master_byte_ctrl.v:453.11-453.68 in step 9
161 summary: reached cover statement i2c_master_byte_ctrl.cov2_stop at i2c_master_byte_ctrl.v:618.4-618.42 in step 9
162 summary: cover trace: i2c_master_cover/engine_0/trace1.vcd
163 summary: reached cover statement i2c_master_byte_ctrl.cov_stop2idle at i2c_master_byte_ctrl.v:457.4-457.66 in step 27
164 DONE (PASS, rc=0)

```

Fig. 20. Cover – Stop Conditions, “PASS” Log.

```

i2c_master.sby X i2c_master.sby#bmc.log X i2c_master.sby#cover.log X
333 engine_0: ## 0:01:15 Writing trace to VCD file: engine_0/trace4.vcd
334 engine_0: ## 0:01:19 Writing trace to Verilog testbench: engine_0/trace4.tb.v
335 engine_0: ## 0:01:19 Writing trace to constraints file: engine_0/trace4.smtc
336 engine_0: ## 0:01:19 Writing trace to Yosys witness file: engine_0/trace4.yw
337 engine_0: ## 0:01:22 Status: passed
338 engine_0: finished (returncode=0)
339 engine_0: Status returned by engine: pass
340 summary: Elapsed clock time [H:MM:SS (secs)]: 0:01:23 (83)
341 summary: Elapsed process time [H:MM:SS (secs)]: 0:01:36 (96)
342 summary: engine_0 (smtbmc --nopsat --unroll --keep-going) returned pass
343 summary: cover trace: i2c_master_cover/engine_0/trace0.vcd
344 summary: reached cover statement i2c_master_byte_ctrl.str_scl_cover at i2c_master_byte_ctrl.v:513.13-513.59 in step 21
345 summary: cover trace: i2c_master_cover/engine_0/trace1.vcd
346 summary: reached cover statement i2c_master_byte_ctrl.cov_write at i2c_master_byte_ctrl.v:547.4-547.108 in step 29
347 summary: reached cover statement i2c_master_byte_ctrl.state_cov_start2write at i2c_master_byte_ctrl.v:542.4-543.59 in
348 summary: cover trace: i2c_master_cover/engine_0/trace2.vcd
349 summary: reached cover statement i2c_master_byte_ctrl.st_cov_write2write at i2c_master_byte_ctrl.v:553.4-553.105 in st
350 summary: cover trace: i2c_master_cover/engine_0/trace3.vcd
351 summary: reached cover statement i2c_master_byte_ctrl.st_cov_write2ack at i2c_master_byte_ctrl.v:558.4-558.99 in step
352 summary: cover trace: i2c_master_cover/engine_0/trace4.vcd
353 summary: reached cover statement i2c_master_byte_ctrl.st_cov_acktoidle at i2c_master_byte_ctrl.v:563.4-563.72 in step
354 DONE (PASS, rc=0)

```

Fig. 21. Cover – Protocol Checking Start, “PASS” Log.

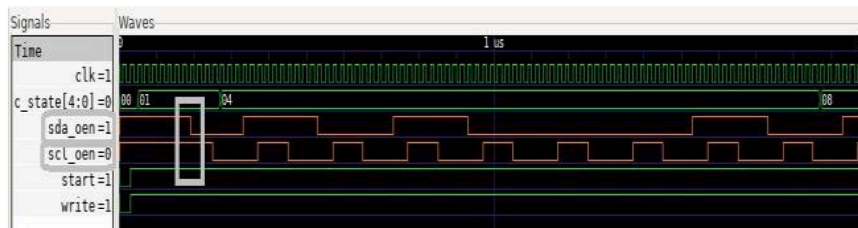


Fig. 22. Waveform Result – Protocol Checking Start.

sequencing. After these refinements, the Prove run returned a “PASS” result for the WRITE state.

6.3.1. Start State

According to the FSM specifications, all ‘ST_START’ states, including the ‘Read/write bit set?’, ‘Start to Read’, and ‘Start to Write’ conditions, have been mathematically verified using the theorem-proving method, which uses induction to reason about behavior beyond the limited depth of the BMC method.

6.3.2. Write State

According to the specifications in the FSM, theorem proving validated the “WRITE to WRITE” and “WRITE to ACK” transitions within the configured bounds. After adjusting assumptions to eliminate an initial UNKNOWN result, PASS was obtained for both transitions; a deliberately inverted assertion produced FAIL.

“PASS”, possible “FAIL” and “UNKNOWN” statuses for the “WRITE to ACK” state are provided. “PASS” result is achieved with the assertion “if(v_clk_cnt_ack > 2) assert (c_state == ST_ACK)”.

When the condition “c_state” is asserted to be not equal to “ST_ACK” a “FAIL” result is obtained, indicating that this assertion is incorrect the

```

150 engine_0: ## 0:00:02 Writing trace to Yosys witness file: engine_0/trace1.yw
151 engine_0: ## 0:00:02 Checking cover reachability in step 25..
152 engine_0: ## 0:00:02 Checking cover reachability in step 26..
153 engine_0: ## 0:00:02 Checking cover reachability in step 27..
154 engine_0: ## 0:00:02 Reached cover statement at cov_stop2idle in step 27.
155 engine_0: ## 0:00:02 Writing trace to VCD file: engine_0/trace2.vcd
156 engine_0: ## 0:00:03 Writing trace to Verilog testbench: engine_0/trace2.tb.v
157 engine_0: ## 0:00:03 Writing trace to constraints file: engine_0/trace2.smtc
158 engine_0: ## 0:00:03 Writing trace to Yosys witness file: engine_0/trace2.yw
159 engine_0: ## 0:00:03 Status: passed
160 engine_0: finished (returncode=0)
161 engine_0: Status returned by engine: pass
162 summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:04 (4)
163 summary: Elapsed process time [H:MM:SS (secs)]: 0:00:05 (5)
164 summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass
165 summary: cover trace: i2c_master_cover/engine_0/trace0.vcd
166 summary: reached cover statement i2c_master_byte_ctrl.cov1_stop at i2c_master_byte_ctrl.v:452.11-452.68 in step 9
167 summary: reached cover statement i2c_master_byte_ctrl.cov2_stop at i2c_master_byte_ctrl.v:622.4-622.42 in step 9
168 summary: cover trace: i2c_master_cover/engine_0/trace1.vcd
169 summary: reached cover statement i2c_master_byte_ctrl.stp_scl_cover at i2c_master_byte_ctrl.v:518.13-518.59 in step 25
170 summary: cover trace: i2c_master_cover/engine_0/trace2.vcd
171 summary: reached cover statement i2c_master_byte_ctrl.cov_stop2idle at i2c_master_byte_ctrl.v:456.4-456.66 in step 27
172 DONE (PASS, rc=0)

```

Fig. 23. Cover – Protocol Checking Stop, “PASS” Log.



Fig. 24. Waveform Result – Protocol Checking Stop.

"FAIL" result is obtained with the assertion "if(v_clk_cnt_ack > 2) assert (c_state != ST_ACK)". In reality, the system is in the "ST_ACK" state, as expected, and the waveform also shows that "c_state" corresponds to 'h08 which represents "ST_ACK" after 'h04 representing "ST_WRITE".

For the WRITE state, the behavior described above is illustrated in Fig. 25, which shows the initial UNKNOWN result (base case PASS, induction FAIL) produced by the Prove mode. Figs. 26 and 27 then present the refined configuration and the corresponding waveform after the constraints and Prove-task setup were adjusted, together with the final PASS outcome for the WRITE state.

6.3.3. Read State

According to the FSM specifications, all ST_READ states were verified using the Theorem Proving method. Both 'READ to READ' and 'READ to ACK' passed within 400 steps, with the 'READ to ACK' "PASS" result shown in Fig. 28.

6.3.4. ACK to IDLE State

According to the FSM specifications, Theorem Proving verified the 'ACK to IDLE' state within 400 steps, resulting in a "PASS" provided similarly to Fig. 28.

6.3.5. Stop State

According to the specifications, after all condition checked, Theorem Proving Method was applied to verify the 'STOP' states within 150 steps. The "PASS" result has been proved as shown in the Fig. 29.

7. Conclusion

The accurate operation and reliable data communication of the I²C Master design, critical for aerospace applications, were evaluated using a design-specific verification framework that can be applicable across all stages of the verification process. In contrast to conventional simulation-

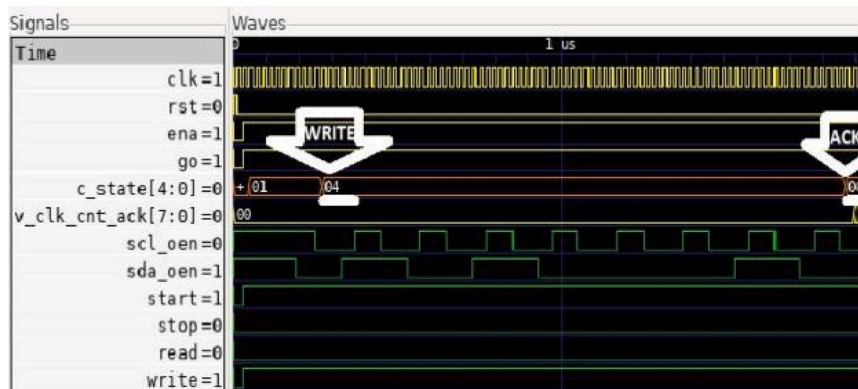


Fig. 25. Waveform Result-Proving-Write to Ack.


```

SBY 23:55:59 [i2c_master_prove] engine_0.basecase: ## 0:05:28 Checking assertions in step 394..
SBY 23:56:01 [i2c_master_prove] engine_0.basecase: ## 0:05:29 Checking assertions in step 395..
SBY 23:56:02 [i2c_master_prove] engine_0.basecase: ## 0:05:31 Checking assertions in step 396..
SBY 23:56:03 [i2c_master_prove] engine_0.basecase: ## 0:05:32 Checking assertions in step 397..
SBY 23:56:05 [i2c_master_prove] engine_0.basecase: ## 0:05:33 Checking assertions in step 398..
SBY 23:56:06 [i2c_master_prove] engine_0.basecase: ## 0:05:35 Checking assertions in step 399..
SBY 23:56:08 [i2c_master_prove] engine_0.basecase: ## 0:05:36 Status: passed
SBY 23:56:08 [i2c_master_prove] engine_0.basecase: finished (returncode=0)
SBY 23:56:08 [i2c_master_prove] engine_0.basecase: Status returned by engine for basecase: pass
SBY 23:56:08 [i2c_master_prove] summary: Elapsed clock time [H:MM:SS (secs)]: 0:05:37 (337)
SBY 23:56:08 [i2c_master_prove] summary: Elapsed process time [H:MM:SS (secs)]: 0:08:40 (520)
SBY 23:56:08 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass for basecase
SBY 23:56:08 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned FAIL for induction
SBY 23:56:08 [i2c_master_prove] summary: counterexample trace [induction]: i2c_master_prove/engine_0/trace_induct.vcd
SBY 23:56:08 [i2c_master_prove] summary: failed assertion i2c_master_byte_ctrl.st_ast_write2ack at i2c_master_byte_ctrl
SBY 23:56:08 [i2c_master_prove] DONE (UNKNOWN, rc=4)

```

Fig. 26. Proving – Write to Ack Cond., “UNKNOWN” Log.



Fig. 27. Waveform Result – Write to Ack Condition.

```

SBY 23:09:06 [i2c_master_prove] engine_0.basecase: ## 0:03:53 Checking assertions in step 397..
SBY 23:09:08 [i2c_master_prove] engine_0.basecase: ## 0:03:55 Checking assertions in step 398..
SBY 23:09:09 [i2c_master_prove] engine_0.basecase: ## 0:03:56 Checking assertions in step 399..
SBY 23:09:10 [i2c_master_prove] engine_0.basecase: ## 0:03:57 Status: passed
SBY 23:09:10 [i2c_master_prove] engine_0.basecase: finished (returncode=0)
SBY 23:09:10 [i2c_master_prove] engine_0.basecase: Status returned by engine for basecase: pass
SBY 23:09:10 [i2c_master_prove] summary: Elapsed clock time [H:MM:SS (secs)]: 0:03:58 (238)
SBY 23:09:10 [i2c_master_prove] summary: Elapsed process time [H:MM:SS (secs)]: 0:04:31 (271)
SBY 23:09:10 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass for basecase
SBY 23:09:10 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass for induction
SBY 23:09:10 [i2c_master_prove] summary: engine_0 did not produce any traces
SBY 23:09:10 [i2c_master_prove] summary: successful proof by k-induction.
SBY 23:09:10 [i2c_master_prove] DONE (PASS, rc=0)

```

Fig. 28. Proving Method – Read to Ack Cond., “PASS”.

```

276 SBY 16:52:54 [i2c_master_prove] engine_0.basecase: ## 0:00:26 Checking assertions in step 147..
277 SBY 16:52:54 [i2c_master_prove] engine_0.basecase: ## 0:00:26 Checking assertions in step 148..
278 SBY 16:52:54 [i2c_master_prove] engine_0.basecase: ## 0:00:26 Checking assertions in step 149..
279 SBY 16:52:55 [i2c_master_prove] engine_0.basecase: ## 0:00:26 Status: passed
280 SBY 16:52:55 [i2c_master_prove] engine_0.basecase: finished (returncode=0)
281 SBY 16:52:55 [i2c_master_prove] engine_0.basecase: Status returned by engine for basecase: pass
282 SBY 16:52:55 [i2c_master_prove] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:28 (28)
283 SBY 16:52:55 [i2c_master_prove] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:34 (34)
284 SBY 16:52:55 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass for basecase
285 SBY 16:52:55 [i2c_master_prove] summary: engine_0 (smtbmc --nopresat --unroll --keep-going) returned pass for induction
286 SBY 16:52:55 [i2c_master_prove] summary: engine_0 did not produce any traces
287 SBY 16:52:55 [i2c_master_prove] summary: successful proof by k-induction.
288 SBY 16:52:55 [i2c_master_prove] DONE (PASS, rc=0)

```

Fig. 29. Waveform Result-Proving- Stop Condition.

based or hybrid hardware/software approaches, the proposed method applies advanced model-checking techniques—bounded model checking, cover analysis, and theorem proving—to systematically explore the relevant state space of the design within the configured bounds.

The verification campaign achieved full coverage of all five FSM states and nine transitions, with all specified properties satisfied and reported as PASS. Potential FAIL and UNKNOWN behaviors were also exercised by deliberately injecting faulty conditions, which allowed us to assess both the robustness of the design and the diagnostic capabilities of the formal flow. By applying assert, cover, and proof directives directly to the DUT outputs and internal control signals, the framework confirmed state reachability and progression and provided

mathematical evidence that the design conforms to the specified properties.

This methodology offers a level of exhaustiveness that simulation-based assertion frameworks alone cannot guarantee, making it particularly suitable for safety-critical avionics systems. Although AI-assisted assertion generation is an attractive research direction, current limitations in semantic understanding and domain specificity led us to define the properties manually in this study to ensure correctness and traceability.

Future work will benchmark the proposed formal verification flow against alternative verification strategies and will investigate AI-assisted property extraction and refinement to further improve automation,

scalability, and coverage for complex avionics communication protocols.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] V. Subedha, S. Sridhar, An efficient coverage driven functional verification system based on genetic algorithm, *European Journal of Scientific Research* 81 (4) (2012) 533–542.
- [2] M. Sukhanya, K. Gavaskar, Functional verification environment for I2C master controller using system verilog, in: 4th International Conference on Signal Processing, Communications and Networking (ICSCN -2017), Chennai, India, 2017, pp. 1–6, <https://doi.org/10.1109/ICSCN.2017.8085732>.
- [3] P. Mulani, J. Patoliya, H. Patel, D. Chauhan, Verification of I2C DUT using system Verilog, *International Journal of Advanced Engineering Technology* 1 (3) (2010) 130–134.
- [4] T.T. Kumar, C.Y. Gopinath, Verification of I2C master core using system Verilog-UVM, *International Journal of Science and Research* 3 (6) (2014) 2042–2046.
- [5] G. Renuka, Monitoring the state of materials in verification environment for IP architectures using python based verification mechanism, *Materials Today: Proceedings* 81 (16) (2023) 761–770.
- [6] I. Traore, D.B. Aredo, Enhancing structured review with model-based verification, *IEEE Transactions On Software Engineering* 30 (11) (2004) 736–753, <https://doi.org/10.1109/TSE.2004.86>.
- [7] Dae-Won Moon, Seung-Hyun Pyo, Dae-Ki Hong, Otgonbayar Bataa, Erdenekhuu Norinpel, Assertion-Based Verification of I2C Module Using SystemVerilog, *Electronics* 14 (8) (2025), <https://doi.org/10.3390/electronics14081687>.
- [8] D. Schwyn, Z. Liu, T. Roscoe, Efeu: generating efficient, verified, hybrid hardware/software drivers for I2C devices, in: EuroSys '25: Proceedings of the Twentieth European Conference on Computer Systems, 2025, pp. 76–93, <https://doi.org/10.1145/3689031.3696093>.
- [9] Y. Bai, G. Bany Hamad, S. Suhaib, H. Ren, "Assertion Forge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL", *arXiv preprint*, 2025, 2025, [doi: 10.48550/arXiv.2503.19174](https://doi.org/10.48550/arXiv.2503.19174).
- [10] M. Kang, M. Liu, G. B. Hamad, S. Suhaib, H. Ren, "FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware", 2024, [doi: 10.48550/arXiv.2410.23299](https://doi.org/10.48550/arXiv.2410.23299).
- [11] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, Z. Xie, H. Zhang, AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs, in: ASPDAC '25: Proceedings of the 30th Asia and South Pacific Design Automation Conference, 2025, pp. 614–621, <https://doi.org/10.1145/3658617.3697756>.
- [12] K. Devarajegowda, L. Servadei, Z. Han, M. Werner, W. Ecker, Formal verification methodology in an industrial setup, in: 22nd Euromicro Conference on Digital System Design (DSD), Kallithea, Greece, 2019, pp. 610–614, <https://doi.org/10.1109/DSD.2019.00094>.
- [13] L. Shen, D. Mu, G. Cao, A. Qin, J. Zhu, W. Hu, Accelerating hardware security verification and vulnerability detection through state space reduction, *Computers & Security* 103 (2020) 1–16, <https://doi.org/10.1016/j.cose.2020.102167>.
- [14] A.F. Fathabadi, M. Dalvandi, M. Butler, B.M. Al-Hashimi, Verifying cross-layer interactions through formal model-based assertion generation, *IEEE Embedded Systems Letters* 12 (3) (2020) 83–86, <https://doi.org/10.1109/LES.2019.2955316>.
- [15] D. Trond (2011) "oc-i2c: OpenCores I2C Core", Github [Online]. Available: <https://github.com/trond/oc-i2c>.
- [16] M. Siegel, Achieving earlier verification closure using advanced formal verification, in: *Formal Methods in Computer Aided Design*, Lugano, Switzerland, 2010, p. 275, 275.
- [17] Jannes Harder, Github [Online], Available: <https://github.com/YosysHQ/sby>, 2022. DoA. 24.03.2022.
- [18] A. Dudekula, S. Mrudula, K. Geetha, R. Rao, Verilog based behavioral modeling multi master I2C bus controller, *International Research Journal of Engineering and Technology (IRJET)* 4 (5) (2017), 2018–2021.
- [19] D. Kaith, J.B. Patel, N. Gupta, An introduction to functional verification of I2C protocol using UVM, *International Journal of Computer Applications* 121 (13) (2015) 10–14, <https://doi.org/10.5120/21599-4703>.
- [20] S.B.Y. YosysHQ, Symbiosys Documentation [Online]. Available: https://symbiyosys.readthedocs.io/_/downloads/en/latest/pdf/, 2023. DoA. 01.10.2023.
- [21] S. Chalana, S. Mitra, S. Bharath, R. Bhimireddy, S.K. Manickam, S. Kumar, Enhancing coverage of clock domain crossing assertion verification leveraging formal, in: *IEEE Women in Technology Conference (WINTeCHCON)*, Bangalore, India, 2023, pp. 1–6, <https://doi.org/10.1109/WINTeCHCON58518.2023.10277198>.
- [22] T. Zhang, D.J. Saab, A. Abraham, Automatic assertion generation for simulation, formal verification and emulation, in: *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Bochum, Germany, 2017, pp. 471–476, <https://doi.org/10.1109/ISVLSI.2017.88>.
- [23] P. Bhamidipati, S.M. Achyutha, R. Vemuri, Security analysis of a system-on-chip using assertion-based verification, in: *IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, Lansing, MI, USA, 2021, pp. 826–831, <https://doi.org/10.1109/MWSCAS47672.2021.9531916>.



Merve Berik received the M. Sc. degree from the Department of Electrical Electronic Engineering, Çankaya University, Ankara, Turkey. She graduated with first-class honors from the Bachelor's program in Electrical Electronic Engineering at Bülent Ecevit University, Zonguldak, Turkey. She currently working as a Lead Digital Design Verification Engineer at Turkish Aerospace-TUSAS, Ankara, Turkey.



Yahya Kemal Baykal received the Ph.D. degree in 1982 from the Department of Electrical Engineering and Computer Sciences, Northwestern University, Evanston, IL, USA. He is currently a Professor with the Department of Electrical-Electronics Engineering, Çankaya University, Ankara, Turkey. He is the author of 245 SCI journal papers and more than 310 publications. His research interests include optical communication, optical wave propagation in atmospheric, underwater media, and telecommunications infrastructure.